Contrato dos domain services â€” patients, sessions e records

Objetivo
Definir os contratos dos **domain services** do Ethos para os domÃnios centrais:
- `patients`
- `sessions`
- `records`

Este documento complementa o contrato dos repositories e define a camada acima da persistÃªncia, responsÃ¡vel por aplicar regras de negÃ³cio, coordenar operaÃ§Ãµes entre domÃnios e oferecer uma API de domÃnio estÃ¡vel para hooks, pÃ¡ginas e fluxos do frontend.

Papel dos domain services

Os domain services existem para:
- aplicar regras de negÃ³cio
- coordenar mÃºltiplos repositories
- validar operaÃ§Ãµes entre entidades relacionadas
- proteger a UI de detalhes de persistÃªncia
- servir como ponto de entrada do domÃnio para fluxos local-first

Os domain services nÃ£o devem
- renderizar UI
- carregar estado visual de tela
- conhecer detalhes de navegaÃ§Ã£o
- depender diretamente de componentes React
- expor detalhes de IndexedDB, HTTP ou storage especÃfico

RelaÃ§Ã£o entre camadas

Repository
ResponsÃ¡vel por persistÃªncia e consulta.

Domain service
ResponsÃ¡vel por lÃ³gica de domÃnio e orquestraÃ§Ã£o.

Hook/UI
ResponsÃ¡vel por interaÃ§Ã£o de tela e experiÃªncia do usuÃ¡rio.

Fluxo desejado:

```text
UI / Hooks -> Domain Services -> Repositories -> Local DB
                                  -> Remote Adapters (quando necessÃ¡rio e opcional)
```

PrincÃpios

1. Local-first por padrÃ£o
Os services devem assumir que a camada local Ã© a fonte primÃ¡ria do domÃnio.

2. Regras de negÃ³cio no serviÃ§o, nÃ£o no componente
ValidaÃ§Ã£o, coerÃªncia e orquestraÃ§Ã£o devem morar aqui.

3. Contratos orientados ao uso real do produto
Os mÃ©todos devem refletir operaÃ§Ãµes que o produto realmente executa.

4. PreparaÃ§Ã£o para evoluÃ§Ã£o futura
Os contracts devem permitir, no futuro:
- sync
- eventos de domÃnio
- auditoria
- backup
- resoluÃ§Ã£o de conflitos

Sem poluir a primeira implementaÃ§Ã£o.

---

1. Patients Domain Service

Papel do domÃnio
Orquestrar o ciclo de vida do paciente e proteger regras centrais ligadas ao cadastro e estado do paciente.

Interface sugerida

```ts
import type {
  Patient,
  CreatePatientInput,
  UpdatePatientInput,
  PatientListFilters,
} from "./patients-repository";

export interface PatientsDomainService {
  getPatient(id: string): Promise<Patient | null>;
  listPatients(filters?: PatientListFilters): Promise<Patient[]>;
  createPatient(input: CreatePatientInput): Promise<Patient>;
  updatePatient(id: string, input: UpdatePatientInput): Promise<Patient>;
  archivePatient(id: string): Promise<void>;
  restorePatient(id: string): Promise<void>;
  deletePatient(id: string): Promise<void>;
}
```

Regras esperadas

`createPatient`
Deve:
- validar campos obrigatÃ³rios
- normalizar dados relevantes
- impedir criaÃ§Ã£o claramente invÃ¡lida
- criar entidade local inicial coerente

`updatePatient`
Deve:
- validar alteraÃ§Ãµes permitidas
- impedir update inconsistente
- preservar integridade de dados principais

`archivePatient`
Deve:
- arquivar paciente sem quebrar histÃ³rico clÃnico
- preservar relaÃ§Ã£o com sessÃµes e records jÃ¡ existentes
- preferir arquivamento a exclusÃ£o destrutiva

`deletePatient`
Nesta fase, deve ser tratado com cautela.
A tendÃªncia recomendada Ã©:
- evitar exclusÃ£o fÃsica direta
- exigir polÃtica explÃcita do produto

---

2. Sessions Domain Service

Papel do domÃnio
Orquestrar o ciclo de vida da sessÃ£o clÃnica e garantir coerÃªncia com o paciente relacionado.

Interface sugerida

```ts
import type {
  Session,
  CreateSessionInput,
  UpdateSessionInput,
  SessionListFilters,
```

```
} from "./sessions-repository";

export interface SessionsDomainService {
  getSession(id: string): Promise<Session | null>;
  listSessions(filters?: SessionListFilters): Promise<Session[]>;
  listSessionsByPatient(patientId: string): Promise<Session[]>;
  createSession(input: CreateSessionInput): Promise<Session>;
  updateSession(id: string, input: UpdateSessionInput): Promise<Session>;
  completeSession(id: string): Promise<Session>;
  cancelSession(id: string): Promise<Session>;
  markSessionNoShow(id: string): Promise<Session>;
  deleteSession(id: string): Promise<void>;
}
```

Regras esperadas

`createSession`
Deve:
- validar se `patientId` existe
- criar sessÃ£o com status coerente
- aplicar defaults corretos

`updateSession`
Deve:
- validar transiÃ§Ãµes de estado
- impedir inconsistÃªncias temporais graves
- manter integridade do vÃnculo com paciente

`completeSession`
Deve:
- atualizar status para concluÃda
- preparar terreno para criaÃ§Ã£o ou atualizaÃ§Ã£o de record relacionado
- ser tratada como operaÃ§Ã£o de domÃnio, nÃ£o simples update cru

`cancelSession`
Deve:
- registrar estado de cancelamento de forma consistente
- evitar perda acidental de histÃ³rico

`markSessionNoShow`
Deve:
- representar falta de forma explÃcita
- nÃ£o confundir com cancelamento ou conclusÃ£o

---

3. Records Domain Service

Papel do domÃnio
Orquestrar o ciclo de vida do prontuÃ¡rio e dos registros clÃnicos, inclusive sua relaÃ§Ã£o com pacientes e sessÃµes.

Interface sugerida

```ts
import type {
  ClinicalRecord,
  CreateRecordInput,
  UpdateRecordInput,
  RecordListFilters,
} from "./records-repository";

export interface RecordsDomainService {
  getRecord(id: string): Promise<ClinicalRecord | null>;
  listRecords(filters?: RecordListFilters): Promise<ClinicalRecord[]>;
  listRecordsByPatient(patientId: string): Promise<ClinicalRecord[]>;
  listRecordsBySession(sessionId: string): Promise<ClinicalRecord[]>;
  createRecord(input: CreateRecordInput): Promise<ClinicalRecord>;
  updateRecord(id: string, input: UpdateRecordInput): Promise<ClinicalRecord>;
  createSessionRecordDraft(sessionId: string, patientId: string): Promise<ClinicalRecord>;
```

```
  deleteRecord(id: string): Promise<void>;
}
```

Regras esperadas

`createRecord`
Deve:
- validar vÃnculo com paciente
- validar vÃnculo com sessÃ£o quando fornecido
- criar registro com metadados corretos

`updateRecord`
Deve:
- versionar adequadamente
- atualizar timestamps corretos
- proteger conteÃºdo clÃnico contra operaÃ§Ãµes inconsistentes

`createSessionRecordDraft`
Deve:
- existir como operaÃ§Ã£o de domÃnio de primeira classe
- permitir iniciar prontuÃ¡rio a partir de uma sessÃ£o
- evitar que a UI tenha que montar esse fluxo sozinha

`deleteRecord`
Deve seguir polÃtica clÃnica conservadora:
- evitar exclusÃ£o destrutiva imediata quando isso comprometer histÃ³rico e rastreabilidade

---

4. OperaÃ§Ãµes transversais sugeridas

Algumas regras exigem coordenaÃ§Ã£o entre domÃnios. Para isso, os services devem poder depender entre si ou usar um service de orquestraÃ§Ã£o superior.

Casos relevantes

Caso A â€” Criar sessÃ£o e preparar prontuÃ¡rio draft
Fluxo:
- valida paciente
- cria sessÃ£o
- cria draft de record vinculado

Caso B â€” Arquivar paciente sem destruir histÃ³rico
Fluxo:
- arquiva paciente
- preserva sessÃµes
- preserva records
- ajusta comportamento de listagem futura

Caso C â€” Concluir sessÃ£o e garantir prontuÃ¡rio consistente
Fluxo:
- conclui sessÃ£o
- garante existÃªncia ou atualizaÃ§Ã£o do record correspondente

---

5. Tipos de erro recomendados

Os domain services devem lanÃ§ar ou retornar erros de domÃnio claros, por exemplo:

```ts
export class NotFoundError extends Error {}
export class ValidationError extends Error {}
export class ConflictError extends Error {}
export class IntegrityError extends Error {}
```

Uso esperado
- `NotFoundError`: entidade inexistente
- `ValidationError`: entrada invÃ¡lida
- `ConflictError`: transiÃ§Ã£o de estado invÃ¡lida
- `IntegrityError`: quebra de vÃnculo entre entidades relacionadas

---

6. O que nÃ£o deve entrar nos services agora

NÃ£o incluir nesta fase, como parte principal dos contracts:
- sync remoto
- upload/download cloud
- analytics
- estado de loading da UI
- lÃ³gica de cache visual
- detalhes de backup
- detalhes de criptografia de baixo nÃvel

Esses pontos pertencem a outras camadas.

---

7. Exemplo de dependÃªncias internas

PatientsDomainService
Depende de:
- `PatientsRepository`

SessionsDomainService
Depende de:
- `SessionsRepository`
- `PatientsRepository` para validaÃ§Ã£o de vÃnculo

RecordsDomainService
Depende de:
- `RecordsRepository`
- `PatientsRepository` para validaÃ§Ã£o de paciente
- `SessionsRepository` para vÃnculo opcional com sessÃ£o

---

8. EstratÃ©gia de implementaÃ§Ã£o

Fase 1
Implementar `PatientsDomainService` e `SessionsDomainService` com foco em:
- criaÃ§Ã£o
- listagem
- atualizaÃ§Ã£o
- validaÃ§Ãµes bÃ¡sicas

Fase 2
Implementar `RecordsDomainService` com foco em:
- prontuÃ¡rio local-first
- vÃnculo com sessÃ£o
- draft inicial

Fase 3
Migrar hooks e pÃ¡ginas para consumir services em vez de falar com repositories ou API diretamente.

---

9. CritÃ©rios de sucesso

Este contrato serÃ¡ considerado adequado se:
- a UI conseguir operar sem depender direto de repository ou API
- regras de domÃnio relevantes deixarem de ficar espalhadas em componentes
- a migraÃ§Ã£o local-first ficar mais previsÃvel
- pacientes, sessÃµes e records puderem evoluir sem forte acoplamento entre tela e persistÃªncia

---

10. PrÃ³ximos artefatos recomendados

Depois deste documento, os mais Ãteis sÃ£o:
1. checklist tÃ©cnico da implementaÃ§Ã£o dos domain services
2. plano de migraÃ§Ã£o dos hooks do frontend
3. plano de migraÃ§Ã£o das pÃ¡ginas centrais para consumo via services
4. schema inicial do banco local alinhado a esses contratos